Handoff 22: Lessons per unit, and browser checks on the lesson page (UI-05)

Date: 2026-10-27 (America/Toronto)
Written by: Claude Code (implementer), in a separate git worktree
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request (PR_NUMBER), not merged, not deployed. **No database migration, no new dependency, no content change** (unit 1 content is untouched; no unit 2).

> **How to use this file:** upload it with `build.md`, `docs/plan/browser-checks.md`, `docs/plan/lesson-feedback.md`, and handoffs 11, 15 and 19. The pull request diff is the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are reviewing Python Academy stage 22 (UI-05). The lesson page now
loads lessons one unit at a time (a new GET /api/units/<unit>/lessons;
/api/lessons is unchanged), so a unit 2 can be added without page
changes. For lessons whose challenge has a browser check (ENG-03), the
page runs the test cases in the learner's browser with check-runner.js
and sends only the reported output to the API's judge, then shows the
result through the ENG-04 feedback. I'm attaching build.md,
docs/plan/browser-checks.md, docs/plan/lesson-feedback.md and this
handoff. Check that unit 1 cannot have changed, that hidden expected
output never reaches the browser, that the API still never runs learner
code, that only "passed" counts as a pass, and that the old and new page
and API work together during a deploy. Explain at a first-year computer
engineering level.
```

1. Step 0 (checks before building)

Check	Result
`git fetch --all`, `gh pr list --state all`	Done; no open PRs
PR #17 (ENG-03) merged?	**Yes**, 2026-10-27 04:27 UTC

Check	Result
PR #22 (ENG-04 part 1) merged?	**Yes**, 2026-10-27 05:19 UTC
PR #25 (ENG-04 part 2a) merged?	**Yes**
Earlier UI-05 work	**None.** One commit (`b81ff46`, a backlog sync) only mentions UI-05
Where the work happened	A separate git worktree, `../python-academy-ui05`, on a new branch `feat/lesson-units` from `origin/main` at `3263342` (after PR #26). The main folder, where other sessions work, was not switched or changed. Not stacked on any PR

2. Handoff record (build.md template)

Field	Value
Task	UI-05: the lesson page loads lessons per unit; challenges with browser checks run their cases with `check-runner.js` and show the judge's result through the ENG-04 feedback; wrong-output results get rule IDs and tagged-hint support
Human goal	Unit 2 (EDU-04) can be written as content only, with real behaviour checks, and unit 1 learners notice nothing
Relevant sections	`build.md` sections 4, 11 and 12; `browser-checks.md`; `lesson-feedback.md` sections 4 to 7; handoff 19 section 7
Repository and branch	`torontojohn/python-academy`, branch `feat/lesson-units`
Commits	`2d79561` records how unit 1 rendered **before** any change (the baseline the tests compare against); `14370e2` is the change; documents in the commits after it
Implementer / reviewer	Claude Code / the owner and a chat assistant
Non-goals	Unit 2 content, any unit 1 content change, rule IDs for construct failures (section 7), predict-first and measurements (ENG-04c), the backlog and decision log (the owner batches those)
Data and compatibility	No migration. The API only **adds**: a new route, three new fields on its lessons, and a `rule` field on browser-check results. `/api/lessons` is byte-for-byte unchanged, so an old page works with the new API. A new page with an old API falls back to `/api/lessons` (tested)
Deployment status	Not requested

Files changed

File	Why
`backend/app/content.py`	`lesson_view()`: one lesson in the page's shape, shared by `/api/lessons` (unchanged) and the new route. `unit_lessons()`: each released unit's lessons plus `course`, `unit` and `check` (`bounded-story` or `browser-tests`). A hint may be tagged `<lesson>.case.<case id>` when its challenge has a browser check
`backend/app/main.py`	`GET /api/units/{unit_id}/lessons`; 404 for an unknown or coming-soon unit
`backend/app/browser_checks.py`	The judge's answer has a `rule` field: `<lesson>.case.<case id>` for wrong output, otherwise empty
`frontend/app/adventure.tsx`	Loads per unit through the catalog, falling back to `/api/lessons`. Unit 1 keeps its heading, pictures and wording, and later units use the catalog's title and summary. Browser-check challenges run their cases (`runChecks`) and show results through the shared `show()`
`backend/tests/fixtures/lesson_units/` (new, **test only**)	A TEST FIXTURE course with two released units, each with a browser-check lesson (`fixture.add-two` with a hidden case and tagged hints; `fixture.double`). Never in `backend/content/`, never served in production
`frontend/tests/fixtures/lesson-units-api.json` (new)	The real API's answers for that fixture (catalog, units, check plans) and ten recorded judge requests and answers. The browser tests mock the API with it
`frontend/tests/fixtures/unit1-page.snapshot.json` (new)	55 texts of unit 1 (every card, hint step and result) recorded from `main` **before** this change
`backend/tests/test_lesson_units.py` (new, 10 tests)	The recordings equal the real API; additive-only; per-unit loading; rule IDs; XP and badges
`backend/tests/test_sessions.py`	The new route in the route-listing safety test (public, like `/api/lessons`)
`frontend/tests/lesson-units.spec.ts` (new, 16 tests), `frontend/tests/unit1-unchanged.spec.ts` (new, 1 test)	Browser tests (section 6)
`docs/plan/browser-checks.md`, `backend/content/README.md`	The `rule` field, case tags, and the lesson page now running checks
`docs/handoffs/2026-10-27-22-lesson-units.md` and `.pdf`	This handoff

Not edited: `docs/plan/backlog.md`, `docs/plan/decision-log.md` (section 10 lists what to add).

3. How it works

Loading: 1. The page reads the catalog. 2. It finds the released unit that holds the requested lesson. With no lesson, or an unknown one, it uses the first released unit, which is unit 1 today, as before. 3. It asks `GET /api/units/<unit>/lessons`. 4. If the catalog, the route or its answer is missing, it asks `/api/lessons`, exactly as before. That covers an older API during a deploy. 5. The lesson menu lists only that unit's lessons, and counts like "Quest 1 of N" use that unit's size.

A challenge with a browser check (`check: "browser-tests"`): 1. The page asks `GET /api/checks/<lesson>`, the plan. Hidden cases come **without** expected output. 2. `check-runner.js` runs every case in a fresh Pyodide worker, with a 3-second limit per case (from the content), the 10,000-character output cap, and no network. 3. The Output panel shows each **visible** case's output ("With the numbers 2 and 3: 5"), or how it stopped. Hidden cases are not shown. 4. The page posts the code, the first output, and per case only `id`, `status`, `output`, `error_type` and `line` to `/api/check` (preview) or `/api/attempts` (student). This is the existing ENG-03 contract; Python's message stays in the browser. 5. The API compares the outputs and parses the code. It never runs the code. 6. The page shows the result: - `passed`: the badge, and "Other ways to do it". **Only** `passed` **is a pass.** - `needs_work`: the judge's words, for example "For 10 and -4, line 1 should be "6"…". The hint tagged for that case (`fixture.add-two.case.negative`) is offered first. - `execution_error` with a Python error: the ENG-04 explanation from the browser's own message ("Python does not know c yet", "Line 3"), the highlighted line, and "Show Python's message". - `execution_error` from the time limit or the output cap: "Your program kept running… within 3 seconds", or "Too much output… more than 10,000 characters". - `unsupported` ("Not checked") and `service_error` ("Check unavailable", for example when Python cannot start): the judge's words. - The API not answering: the same alert as unit 1, with the code kept.

On the "Try it" card nothing changes: a browser-check lesson runs normally there, with the player replies.

4. Decisions and why

Decision	Options considered	Reason	Who decided
A **new route** per unit, and `/api/lessons` left exactly as it was	Add the per-unit data to `/api/lessons`; reshape it	The page and API deploy separately; leaving the old route untouched is the safest "additive only"	Implementer
The page finds the unit through `/api/catalog`	A new "which unit is this lesson in" route	The catalog already lists units and their lessons; no new data	Implementer
Fall back to `/api/lessons` on any failure of the new path	Show an error	A new page with an old API, or a hiccup, still gives unit 1	Implementer

Decision	Options considered	Reason	Who decided
Unit 1 keeps its own heading, story pictures and "five quests" wording; other units use the catalog's title and summary	Make unit 1 generic too	"Unit 1 must behave exactly as it does today"; its words are written for unit 1 (**assumption 1**: later units get the plain catalog wording until a design for them exists)	Owner rule / implementer
The Output panel shows visible cases only	Show every case	Hidden cases are hidden; their output adds nothing for the learner	Implementer
A Python error is explained from the browser's report (the ENG-04 translation), not the judge's sentence	Show the judge's sentence	`lesson-feedback.md` section 6: an `execution_error` shows the error translation. The judge's sentence is used when there is nothing to translate	Spec
Rule IDs only for **wrong output** (`<lesson>.case.<case id>`)	Rule IDs for every reason	Wrong output is what handoff 19 named, and it fits the content model with a few lines. Construct failures need more (section 7)	Implementer (**assumption 2**)
The judge's `rule` is empty for every other reason, and a missing reply still tags `EOFError`	A rule per reason	The EOFError hint already exists in ENG-04's tag set	Implementer
An API failure shows unit 1's alert, not "Check unavailable"	A new message	Same behaviour in both flows. "Check unavailable" stays for the judge's own `service_error`	Implementer (**assumption 3**)
Recorded real API answers drive the browser tests	Hand-written mocks	A backend test proves every recorded answer is what the API says, so the mocks cannot drift; a recorded answer is only returned for exactly the recorded request	Implementer
The "send learner code to the API" break was interpreted as **sending Python's message**	Removing the code from the request	The ENG-03 contract does send the code (the judge parses it, and attempts store it, as in unit 1). What must never be sent is Python's message, which can echo learner text (**assumption 4**)	Implementer

5. Bugs found and how they were fixed

Bug or near-miss	How it was found	Fix
Older test mocks answer unknown routes with `{}`, which made the page fail instead of falling back. A real proxy or old API could do the same	Reasoning before the full run, then confirmed by it	The page falls back to `/api/lessons` unless the unit answer is a non-empty list
My first mock picked a recorded answer by code alone. "Python could not start" and "passed" use the same code, so the wrong answer came back	The service_error test failed	The mock answers only for exactly the recorded request (code **and** reports); that scenario got its own test, because the browser had cached Pyodide earlier in the same test
Treating `execution_error` as a pass (break B3) was caught by only **one** test	The deliberate-break run	Added "no badge, not passable" checks to the timeout and output-cap tests; now 3 tests catch it
B3, written plainly, would not even build: TypeScript said the comparison could never be true	The build	Good news: the type checker is a guard too. The break was re-written with a cast so the tests could be checked
Near-miss: axe reported colour contrast on card 1	The axe run	The card's existing 0.25-second fade-in (from 50 % opacity): scanned after it ends, 0 violations, on unit 1 too. Not caused by this change
Building with a placeholder recordings file failed the type check	The build	The file now always has its full shape

6. How it was tested

Commands and results (in the worktree, macOS, Python 3.12.14, Node 24)

Command	Result
`cd backend && DATABASE_URL=sqlite:///<temp>.db ../../python-academy/backend/.venv/bin/python -m pytest -q`	**367 passed, 11 skipped**
Same with `TEST_POSTGRES_URL=<local PostgreSQL 16 from pgserver>`	**378 passed**
`cd frontend && npm ci --include=dev && npx tsc --noEmit && npm run build`	OK
`cd frontend && CI=1 npx playwright test`	**125 passed** (108 before, plus 17 new)
axe-core 4.10.3 (WCAG 2.0/2.1/2.2 A and AA plus best practices): unit 2 card 1, a needs-work result with its tagged hint, a Python error, a timeout, a pass with other ways; each at 1280 px and 375 px	**0 violations** (after the card's fade-in; section 5); no sideways scrolling

What the tests cover

Required test	Where
Unit 1 unchanged (snapshot-style)	`unit1-unchanged.spec.ts`: 55 texts, every card of every unit 1 lesson, each hint step, a challenge result and a "Try it" result, compared with the snapshot recorded from `main` before the change (commit `2d79561`). Plus `test_unit_1_per_unit_is_the_original_lessons_with_three_fields_added`
Per-unit loading with a fixture second unit	Browser: `fixture.double` loads only unit `fixture.u02`, with its own title, "Quest 1 of 1", no unit 1 pictures, one units request, and no `/api/lessons`. Unit 1 by default; fallback when the route is missing. Backend: each fixture unit served on its own; real content serves only `py.u01`; 404 for coming-soon and unknown units
A browser-check lesson end to end (real Pyodide)	`lesson-units.spec.ts`, one test per state: **passed**; **needs_work** (with the case-tagged hint first); a **hidden case** failing; **execution_error** (explanation, line 3, Python's message); an **infinite loop** stopped at **3 seconds** (whole run under 30 s); **output over the cap**; **unsupported** (`import turtle`); a missing reply; **service_error** (Python cannot start); **the API failing** (500); a unit 2 lesson passing. Each checks the page posted exactly the recorded request
Hidden expected output never in a response	Browser: every API answer the page received across several scenarios, checked for the hidden expected values. Backend: the same for every recorded answer, the plans carry expected output only for visible cases, and lessons carry no cases. ENG-03's own leak test still passes
A learner who passed before keeps XP and badges, nothing twice	`test_a_learner_who_passed_before_keeps_xp_and_badges_and_gets_nothing_twice`: XP 100 after `hello`; loading per unit changes nothing; passing `hello` again and a browser-check lesson twice gives 200 XP and 2 badges in total
`/api/lessons` changes additive only	`/api/lessons` equals the frozen fixture; every unit 1 lesson per unit equals it plus exactly `course`, `unit`, `check`
Keyboard and axe on the new states	Run the check from the keyboard, then "Go to line 3" with Enter. Phone width and reduced motion. axe as above
The page sends only what the contract allows	Each run sends exactly `id, status, output, error_type, line`; no Python message (`is not defined` never in a request)

Mocks: the browser tests mock the API with recorded **real** answers (proven by `test_the_recorded_answers_are_the_real_apis`). Python, the worker and the runner are real. The fixture lessons are test data.

Deliberate-break controls (each undone; files matched saved copies byte for byte afterwards)

What I broke	Tests that failed
B1. Python's error message is sent to the API (`forSubmission` keeps `message`)	**11**: every scenario that posts runs (each checks the exact request), plus ENG-03's "message stays in the browser"
B2. The check plan includes the hidden cases' expected output	**3**: ENG-03's two hidden-output tests and `test_the_recorded_answers_are_the_real_apis`
B3. `execution_error` counts as a pass	**3** (after strengthening; 1 before): Python error, endless loop, output cap
B4. Unit 1 loses its own heading (treated like any other unit)	**2**: the unit 1 snapshot and "unit 1 loads per unit, exactly as before"

7. Known limitations, and what item 3 still needs

- **Rule IDs exist only for wrong output.** Giving construct failures (a required or forbidden `input()`, `import`, …) their own IDs needs: 1. an ID-safe name for each construct (`input()` cannot be part of an ID; for example `uses-input`, `no-import`); 2. the judge to return `rule: <lesson>.require.<name>` or `<lesson>.forbid.<name>`; 3. `content.feedback_errors` to accept those tags.

That is small, but it is a content-format decision (the names), so it is listed rather than built. Timeouts, the output cap and missing replies use the existing error tags. - **No real lesson uses browser checks yet.** The new path can only be seen with the test fixture until unit 2 (EDU-04) is written. - **The Stop button during a browser check** stops showing the run, but the background runner finishes its current case, at most 3 seconds. - **Later units get plain catalog wording** for the heading (assumption 1). A designed heading per unit is for SITE-03 or EDU-04. - **Not tested with a real screen reader,** or on Safari or Firefox.

8. Live checks for you after merging and deploying

Unit 1 must behave exactly as before. You have one test account; use it, or `/preview` as a parent.

- [ ] **Unit 1 looks the same.** Open "Open the story" and click through all five cards: the same heading ("Build Your Own Adventure."), pictures, wording, the five lessons in the menu and the hint ladder.
- [ ] **It loads per unit.** In DevTools → Network, reload a lesson page. You see `units/py.u01/lessons` (200) and no request to `/api/lessons`.
- [ ] **The new route.** In a terminal:

```
sh curl -s -o /dev/null -w "%{http_code}\n" https://python-academy-dji8.onrender.com/api/units/py.u01/lessons # 200 curl -s -o /dev/null -w "%{http_code}\n" https://python-academy-dji8.onrender.com/api/units/py.u02/lessons # 404, coming soon curl -s https://python-academy-dji8.onrender.com/api/lessons | head -c 60 # starts [{"id":"hello" as before
```

- [ ] **Results unchanged.** On card 5 of "Open the story", run `print("hi")` then `print("woah")`: "Check passed…That part is your call." Run `print("hi")` twice: "Not yet. Both lines say the same thing…".
- [ ] **XP unchanged.** As your test student, note the XP and badges on the home page, pass a lesson you already passed, and reload: the same XP and badges.
- [ ] **No CSP reports** in DevTools → Console while doing the above.
- [ ] **The browser-check path** cannot be checked live yet: no real lesson has a browser check. It will be when unit 2 ships.

9. Questions for the owner

1. **Assumption 1:** later units show their catalog title and summary as the heading. OK until unit 2 has its own design?
2. **Assumption 2:** rule IDs for wrong output only, now. Do you want construct rule IDs (section 7) with unit 2?
3. **Assumption 3:** an API failure during a browser check shows the same alert as unit 1. OK?
4. **Assumption 4:** "send learner code to the API" read as "send Python's error message", because the ENG-03 contract sends the code by design. Right?

10. For your docs PR (not edited here)

Where	What
Backlog UI-05	proposed → in_review (PR_NUMBER, handoff 22)
Backlog, new row	"Construct rule IDs for browser checks" (section 7), proposed, with or before EDU-04
Stage table	"22. UI-05 | Lessons per unit; browser checks on the lesson page | in_review (PR_NUMBER, handoff 22)"
Decision log	Chapter 18 from sections 4 to 6 of this handoff

11. Next concrete action

Review and merge, run the live checks, then EDU-04 (unit 2) can be written as content with browser checks.

12. Removing the worktree safely

Everything in `/Users/DEV/Projects/python-academy-ui05` is committed and pushed; only ignored files (`node_modules`, `.next`, caches) are left. From the **main** folder:

```
cd /Users/DEV/Projects/python-academy
git -C ../python-academy-ui05 status --short   # must print nothing
git worktree remove ../python-academy-ui05     # deletes only that folder; refuses if
anything is uncommitted
git worktree prune
```

After PR PR_NUMBER is merged, `git branch -d feat/lesson-units` deletes the local branch; `-d` refuses if it is not merged. Don't delete the folder in the Finder or with `rm -rf`: Git would keep a stale record of it (`git worktree prune` fixes that). The FB-04 worktree (`../python-academy-fb04`) can be removed the same way once PR #26 is merged, which it now is.